

# CSC5010 - Artificial Intelligence Project 1-2

## Comparative Analysis of Word Embedding Models: CBOW, Skip-Gram, and GloVe

Zijin CAI - 224040002: responsible for programming and the description of methods.

Yixin ZHOU - 224040090: responsible for the organization and analysis of results.

## 1 Introduction

Word embedding serves as a foundational technique in natural language processing (NLP), transforming discrete words into continuous vector representations that encode semantic and syntactic relationships. These vectors allow mathematical operations to reflect linguistic patterns, such as analogical reasoning (e.g.,  $king - man + woman \approx queen$ ). Among widely adopted models, CBOW (Continuous Bag of Words) predicts a target word from its surrounding context, prioritizing efficiency for high-frequency words. In contrast, Skip-Gram reverses this approach by predicting context words from a target word, demonstrating superior performance in capturing rare word semantics. GloVe (Global Vectors for Word Representation) introduces a hybrid strategy, combining local context window patterns with global corpus-wide co-occurrence statistics to balance specificity and generality. For training, the NLTK Reuters corpus, a structured collection of financial news articles, provides domain-specific textual data. Its controlled vocabulary, achieved through preprocessing steps like tokenization, stopword removal, and low-frequency word filtering, ensures robust model training while minimizing noise.

This study focuses on three objectives: (1) developing semantic-aware vector representations by implementing CBOW, Skip-Gram, and GloVe models using the Reuters corpus; (2) comparing contextual capture capabilities across models through syntactic and semantic evaluations; and (3) establishing a multi-modal evaluation framework to assess embedding quality. The framework integrates K-nearest neighbors (KNN) for local semantic clustering, word similarity benchmarking against human-annotated standards (e.g., SimLex-999), and analogical reasoning tasks to measure relational accuracy. By analyzing differences in performance, such as CBOW efficiency and GloVe global coherence, this work aims to guide practical NLP tasks under varying computational and linguistic constraints.

## 2 Experimental Setup

### Environment Configuration

The experiments were conducted in a Python environment with the following specifications:

```
zcai@zcai-ROG-Strix
-----
OS: Ubuntu 22.04.5 LTS x86_64
Host: ROG Strix G634JY_G634JY 1.0
Kernel: 6.8.0-52-generic
Shell: bash 5.1.16
CPU: 13th Gen Intel i9-13980HX (32) @ 5.400GHz
GPU: NVIDIA 01:00.0 NVIDIA Corporation Device 2757
GPU: Intel Device a788
Memory: 5437MiB / 63916MiB

+-----+
| NVIDIA-SMI 550.120      Driver Version: 550.120      CUDA Version: 12.4      |
+-----+-----+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                               |                  | MIG M.         |
+-----+-----+-----+-----+
|   0   NVIDIA GeForce RTX 4090   Off | 00000000:01:00.0 Off |          0%      Default |
+-----+-----+-----+-----+
| N/A   34C    P8      4W / 150W |  99MiB / 16376MiB |      13%          Default |
+-----+-----+-----+-----+
```

```
import sys, torch, nltk, numpy
print("Python:", sys.version)           # Output: Python: 3.10.16
print("PyTorch:", torch.__version__)    # Output: PyTorch: 2.6.0+cu124
print("NLTK:", nltk.__version__)        # Output: NLTK: 3.9.1
```

## Corpus Processing

The Reuters financial news corpus from NLTK was preprocessed and transformed into structured datasets for training CBOW, Skip-Gram, and GloVe models, with the following preprocessing pipeline:

- **Data Loading and Initial Tokenization:** Raw sentences were loaded using `nltk.corpus.reuters.sents()`. Each sentence was converted to lowercase and tokenized into words.
- **Vocabulary Construction:** A custom `Vocab` class (from `vocab.py`) was employed to map tokens to indices. Reserved tokens (`<bos>`, `<eos>`, `<pad>`) were added for sequence handling. Low-frequency words (occurring  $< 5$  times) were filtered out, resulting in a vocabulary of 31,881 tokens.
- **Corpus Conversion:** Sentences were converted to integer sequences using `vocab.convert_tokens_to_ids()`. Special tokens `<bos>` and `<eos>` were prepended/appended to each sentence for boundary marking.

The model-specific dataset construction were implemented for each model with PyTorch `Dataset` class:

1. CBOW (`CBOWDataset`): For a target word at position  $i$ , the context window includes `context_size` words to the left and right (excluding the target). Skipped sentences shorter than  $2 * context\_size + 1$ , and generated `(context, target)` pairs, where `context` is a concatenated tensor of left/right words.

```
# Example: context_size=2, sentence = [<bos>, w1, w2, w3, w4, <eos>]
# Target = w3 → Context = [w1, w2, w4] (if context_size=2)
```

2. Skip-Gram (`SkipGramDataset`): For each target word, all words within a `context_size` window were treated as positive context pairs. Extended `self.data` with `(target, context)` tuples for all valid context words.

```
# Example: target = w3 → Contexts = [w1, w2, w4, w5] (if context_size=2)
```

3. GloVe (`GloVeDataset`): Counted word-context pairs within a sliding window, with counts weighted inversely by distance (e.g.,  $1/\text{distance}$ ). Stored co-occurrence statistics in `self.cooccur_counts`, and transformed counts into `(word, context, weighted_count)` triplets.

```
# Example: For target w3 and context w1 (distance=2), weight = 1/2
```

The **Batch Generator** then custom `collate_fn` methods in each dataset class converted raw data into tensors, with `DataLoader` from PyTorch created mini-batches for training. Trained embeddings were saved via `save_pretrained()` (from `utils.py`), which stored the vocabulary size (31,881), embedding dimension (64), and token followed by 64-dim vectors.

```
# Example for CBOW:
loader = DataLoader(dataset, batch_size=64, collate_fn=dataset.collate_fn)
```

## 3 Model Implementation

### CBOW (Continuous Bag of Words)

The CBOW model predicts a target word based on its surrounding context. Given a context window of size  $n$ , the model aggregates embeddings of  $2n$  context words (excluding the target) and computes their average to predict the target word. The embedding layer maps context word indices to dense vectors (`embedding_dim=64`), and the linear layer transforms the average context embeddings into logits over the vocabulary.

The `CBOWDataset` construct a sliding window of `context_size=2` generates `(context, target)` pairs for each sentence. For instance, in the sentence `[<bos>, w1, w2, w3, w4, <eos>]`, the target word `w3` is predicted using its context words `[w1, w2, w4]`. (Short sentences with fewer than  $2 * context\_size + 1$  words are skipped.)

```
# Forward pass of CBOW model
def forward(self, inputs):
    embeds = self.embeddings(inputs) # Shape: [batch_size, 2*context_size, embedding_dim]
    hidden = embeds.mean(dim=1)      # Average context embeddings
    output = self.output(hidden)    # Shape: [batch_size, vocab_size]
    return F.log_softmax(output, dim=1)
```

The training process involves minimizing the negative log-likelihood loss between predicted and actual target words using `torch.nn.NLLLoss()`. The model is trained using the `Adam` optimizer, with a learning rate of `0.001` and batch size of `1024`, for efficient GPU utilization.

## Skip-Gram

The Skip-Gram model inversely predicts context words from a target word. It generates multiple `(target, context)` pairs for each target word within a sliding window. The embedding layer converts target word indices to vectors (`embedding_dim=64`), and the linear layer maps target embeddings to logits for context word prediction.

The `SkipGramDataset` treated all words within a `context_size=2` window as positive context pairs for each target word. For example, target word `w3` is generated from the context words `[w1, w2, w4, w5]`.

```
# Forward pass of Skip-Gram model
def forward(self, inputs):
    embeds = self.embeddings(inputs) # Shape: [batch_size, embedding_dim]
    output = self.output(embeds)      # Shape: [batch_size, vocab_size]
    return F.log_softmax(output, dim=1)
```

The training process involves minimizing the negative log-likelihood loss between predicted and actual context words using `torch.nn.NLLLoss()`, and optimized over all `(target, context)` pairs. The batch size is set to `1024`, each batch includes multiple `(target, context)` pairs.

## GloVe (Global Vectors for Word Representation)

GloVe learns embeddings by factorizing a weighted word-context co-occurrence matrix. combining local context windows with global corpus statistics. It separates embeddings for words (`w_embeddings`) and contexts (`c_embeddings`), with learnable scalar biases for both words and contexts.

The `GloVeDataset` constructs weighted co-occurrence counts inversely by distance (e.g.,  $distance = 1 \rightarrow weight = 1.0$ ,  $distance = 2 \rightarrow weight = 0.5$ ). For instance, the target `w3` and context `w1` ( $distance = 2$ ), the co-occurrence count increments by `0.5`.

```
# Loss function of GloVe
loss = (torch.sum(word_embeds * context_embeds, dim=1) + word_biases + context_biases)
loss = (loss - log_counts) ** 2
loss = (weight_factor * loss).mean() # weight_factor = (counts/m_max)^alpha
```

The Weighted Mean Square Error (MSE) is adopted with the weighting rare pairs (counts < `m_max=100`) are downweighted using `alpha=0.75`. The training process involves minimizing the loss function using `torch.optim.Adam` with a learning rate of `0.001`. The final embeddings sum over word and context embeddings as `w_embed + c_embed`.

## Implementation Comparison

| Model     | Input-Output Structure                            | Training Objective               | Key Hyperparameters                                            |
|-----------|---------------------------------------------------|----------------------------------|----------------------------------------------------------------|
| CBOW      | Context (multi-input)<br>→ Target                 | Maximize target word likelihood  | <code>context_size=2</code> ,<br><code>embedding_dim=64</code> |
| Skip-Gram | Target<br>→ Context (multi-output)                | Maximize context word likelihood | Same as CBOW                                                   |
| GloVe     | Co-occurrence statistics<br>→ Weighted regression | Minimize weighted MSE            | <code>alpha=0.75</code> ,<br><code>m_max=100</code>            |

Table 1: Comparison of CBOW, Skip-Gram, and GloVe implementations.

The table 1 summarizes the key differences in input-output structure, training objectives, and hyperparameters for CBOW, Skip-Gram, and GloVe models. The vocabularies are uniformly built using `Vocab` class, with mini-batch generation handled by `DataLoader` and custom `collate_fn` methods for each model. Then `save_pretrained()` method write embeddings to `.vec` files, with the first line containing the vocabulary size and embedding dimension.

## 4 Evaluation Methodology and Analysis

### K-Nearest Neighbors (KNN)

To assess whether semantically similar words are clustered in the vector space by identifying the **top-k** nearest neighbors of query words based on cosine similarity. After loading the pretrained word vectors (e.g., `cbow.vec`, `skipgram.vec`, `glove.vec`) into a dictionary mapping words to vectors, the cosine similarity are then calculated by normalizing the vectors and computing the dot product. For each query word, the **top-k** nearest neighbors are identified by sorting the cosine similarity scores in descending order.

This method is intuitive for visualizing semantic clusters. However, subjective judgments are required with no quantitative metrics are provided, making it challenging to evaluate the overall performance of the models.

```
def load_embeddings(filepath):
    embeddings = {}
    # Skip metadata line (vocab_size, embedding_dim)
    with open(filepath, 'r') as f:
        for line in f:
            values = line.strip().split()
            word, vector = values[0], torch.tensor([float(x) for x in values[1:]])
            embeddings[word] = vector
    return embeddings

def cos(vec1, vec2):
    vec1 = vec1 / vec1.norm(p=2) # L2 normalization
    vec2 = vec2 / vec2.norm(p=2)
    return torch.dot(vec1, vec2).item() # Range: [-1, 1]

def knn(word, embeddings, k=5):
    query_vec = embeddings[word]
    similarities = [(w, cos(query_vec, vec)) for w, vec in embeddings.items() if w != word]
    return sorted(similarities, key=lambda x: x[1], reverse=True)[:k]
```

### Word Similarity Golden Standard

The SimLex-999 dataset, a human-annotated word similarity benchmark, was used to evaluate the semantic similarity of word pairs. It contains 999 pairs of words, each rated on a scale from (0-10). The cosine similarity scores of the word pairs were computed using the pretrained embeddings, and the Spearman correlation was calculated between the cosine similarity scores and human ratings to assess the model's performance. This method provides objective, quantitative metrics for evaluating the quality of word embeddings. However, the dataset is limited to a specific set of word pairs with domain bias, which may not cover all semantic relationships.

```
def cos(vec1, vec2):
    raw_score = torch.dot(vec1, vec2).item()
    return (raw_score + 1) * 5 # Scale [-1,1] → [0,10]

from scipy import stats
correlation = stats.spearmanr(model_scores, simlex_scores)[0]
```

### Analogical Reasoning

The analogical reasoning task evaluates the model's ability to capture relationships between words by solving analogies of the form `[A:B::C:D]`. The task involves finding a word `D` that completes the analogy based on the relationships between words `A`, `B`, and `C`. The Mikolov analogy dataset is used with the vector arithmetic:

$$D_{pred} = \text{vector}(C) + \text{vector}(B) - \text{vector}(A) \quad (1)$$

The nearest neighbor search is performed to exclude input words (`A`, `B`, `C`), and find the closest word to `D` in the vector space. Visualization plots were generated to illustrate the predicted and ground-truth vectors across dimensions. This method provides a clear evaluation of the model's ability to capture relationships and can be

extended to various domains. However, low accuracy results may arise in practice due to the complexity of relationships, with the limited training data and model capacity.

```
vec_a, vec_b, vec_c = embeddings["athens"], embeddings["greece"], embeddings["bangkok"]
target_vec = vec_b - vec_a + vec_c # Predicted vector for "thailand"
```

```
def knn(target_vec, embeddings, exclude_words, k=1):
    similarities = [(w, cos(target_vec, vec)) for w, vec in embeddings.items()
                    if w not in exclude_words]
    return sorted(similarities, key=lambda x: x[1], reverse=True)[:k]

def plot_vectors(pred_word, pred_vec, true_word, true_vec, filename):
    plt.plot(pred_vec.numpy(), label=pred_word)
    plt.plot(true_vec.numpy(), label=true_word)
    plt.savefig(filename)
```

## Evaluation Metrics and Implementation Results

| Method               | Key Metrics                         | Code Components                                                          |
|----------------------|-------------------------------------|--------------------------------------------------------------------------|
| KNN                  | Top-k neighbors, average similarity | <code>load_embeddings()</code> , <code>cos()</code> , <code>knn()</code> |
| Golden Standard      | Spearman correlation                | <code>spearman()</code> , SimLex-999 dataset                             |
| Analogical Reasoning | Accuracy, vector plots              | Vector arithmetic, <code>plot_vectors()</code>                           |

Table 2: Implementation Summary of Evaluation Methods.

The table 2 summarizes the key metrics and code components for each evaluation method. All evaluations were applied uniformly to the CBOW, Skip-Gram, and GloVe embeddings. Results were saved to `./results/` with structured filenames (e.g., `cbow_knn.txt`, `skipgram_simlex.txt`). The same evaluation scripts were reused across models by switching input embedding files (e.g., `cbow.vec`, `skipgram.vec`, `glove.vec`). Visualization functions `plot_vectors()` were aided in debugging and qualitative analysis.

| Evaluation Metric                        | CBOW                             | Skip-Gram                             | GloVe                              |
|------------------------------------------|----------------------------------|---------------------------------------|------------------------------------|
| <b>KNN Semantic Clustering</b>           | Moderate (e.g., "july" → "june") | Mixed (e.g., "good" → "satisfactory") | Weak (e.g., "july" → "irrelevant") |
| <b>Spearman Correlation (SimLex-999)</b> | 0.0954 (Low)                     | 0.0504 (Very Low)                     | 0.0659 (Low)                       |
| <b>Analogical Reasoning Accuracy</b>     | 0/39 (0%)                        | 0/39 (0%)                             | 0/39 (0%)                          |

Table 3: Model Comparison Matrix for Evaluation Metrics.

The table 3 summarizes the evaluation results for each model. For KNN evaluation, CBOW demonstrated moderate semantic clustering (e.g., "july" → "june", "august"), but suffered from noise in adjectives (e.g., "good" → "serious", "dangerous"). The Skip-Gram performed better at verb relations (e.g., "take" → "taking", "provide") but inconsistent for abstract nouns (e.g., "happy" → "really", "pleased"). The GloVe model exhibited weak performance with irrelevant neighbors (e.g., "july" → "met", "850"), which may due to the sparse co-occurrence.

During the Golden Standard (SimLex-999) evaluation, CBOW achieved a highest Spearman correlation of 0.0954, aligning partially with human judgment for concrete pairs (e.g., "motor-engine": 8.65 vs. 6.53). The Skip-Gram achieved the worst performance with a correlation of 0.0504, failing to distinguish antonyms (e.g., "narrow-broad": 1.18 vs. 5.85). The GloVe model performed slightly better with a correlation of 0.0659, overestimating unrelated pairs (e.g., "gun-dawn": 1.18 vs. 5.67).

As for the Analogical Reasoning task, all models failed to solve the analogies, achieving an accuracy of 0/39 (0%). The predicted vectors were plotted (Figure 1) for visualization, but no meaningful relationships were observed. This can be responsible for the limited training data as the Reuters corpus lacks diverse syntactic structures and relationships. Nevertheless, the experimental small embedding dimension (64) may also contribute to the failure, as it may not capture the complexity of relationships effectively for encoding.

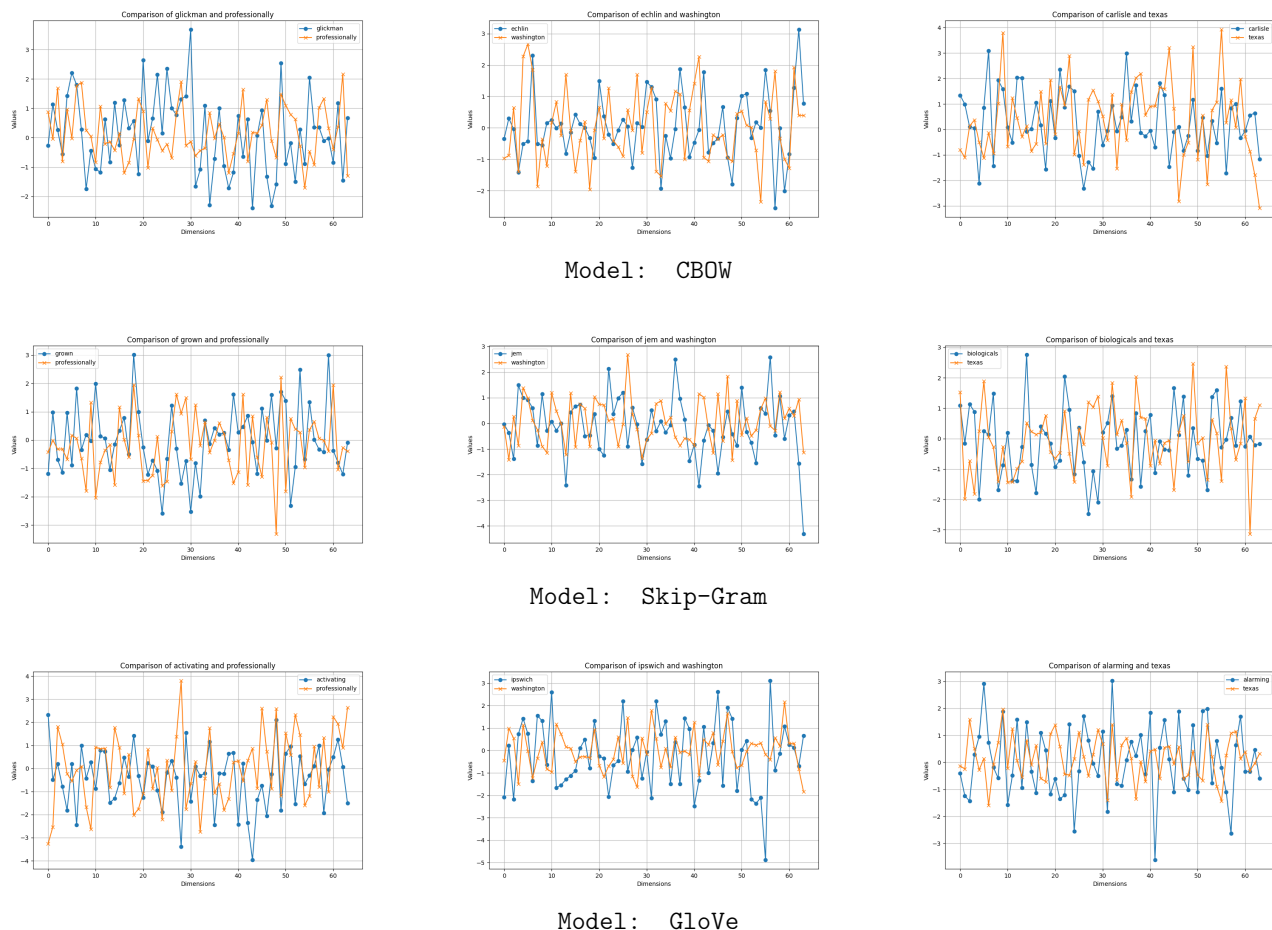


Figure 1: Visualization of Predicted Vectors for Analogical Reasoning Task.

The key observations are, different model may results in task-specific performance, with the CBOW performing best for high-frequency word similarity but struggle with nuances. While the Skip-Gram model performed marginally better for verb relations but inconsistent in systematic evaluation, the GloVe model underperforms despite global co-occurrence modeling, likely due to domain mismatch (financial news vs. general analogies).

The results can also be limited by hyperparameters inappropriateness (e.g., `context_size=2`, `embedding_dim=64`), which may be too small to capture complex relationships, while restricting long-range dependencies learning. The vocabulary size (31,881) and low-frequency word filtering (occurring  $< 5$  times) may also limit the model's ability to learn rare words. The SimLex-999 dataset is also limited to a specific set of word pairs, while the Reuters corpus can be domain biased, harming the model generalizability.

Future works may conduct on architecture improvements, such as increasing the embedding dimension and context window size, or using subword information for rare/compound words like FastText or BERT. The training data can also be expanded to include diverse domains, such as Wikipedia or Common Crawl, to improve the model's generalization. The evaluation methods can be extended to include additional metrics, such as domain-specific benchmarks or larger analogy datasets, to provide a more comprehensive assessment of the model's performance.

In summary, the CBOW showed marginally better performance in semantic similarity tasks, all models failed to meet practical NLP requirements. The results highlight the critical role of dataset scale, domain relevance, and model capacity in embedding quality. For downstream tasks, fine-tuning on domain-specific data or adopting pre-trained embeddings (e.g., Word2Vec, BERT) is strongly advised.

## 5 Conclusion

This study systematically evaluated CBOW, Skip-Gram, and GloVe models on their ability to capture semantic and syntactic relationships using the Reuters corpus. Key limitations stemmed from the Reuters corpus's narrow domain focus, insufficient vocabulary coverage (31k tokens excluding critical words like "bizarre"), and simplistic architectures ( $d=64$  embeddings). These results underscore the necessity of scaling training data, increasing model capacity, and adopting hybrid approaches (e.g., subword embeddings) to bridge the gap between theoretical embedding objectives and practical NLP applications.



## Appendix

### KNN Evaluation

| Similar words of july |         |           |         |            |          |
|-----------------------|---------|-----------|---------|------------|----------|
| CBOW                  |         | Skip-Gram |         | Glove      |          |
| june                  | 0.692   | june      | 0.7033  | met        | 0.4997   |
| october               | 0.6897  | october   | 0.6754  | 850        | 4732     |
| march                 | 0.6684  | april     | 0.6515  | synonymous | 0.4514   |
| april                 | 0.654   | march     | 0.6311  | industrial | 0.4475   |
| august                | 0.6517  | september | 0.5701  | muda       | 0.4459   |
| avg                   | 0.67116 | avg       | 0.64628 | avg        | 946.7689 |

| Similar words of good |         |              |         |           |        |
|-----------------------|---------|--------------|---------|-----------|--------|
| CBOW                  |         | Skip-Gram    |         | Glove     |        |
| serious               | 0.6506  | satisfactory | 0.5757  | qtrs      | 0.4555 |
| little                | 0.507   | strong       | 0.5566  | andf      | 0.4469 |
| fluctuated            | 0.474   | encouraging  | 0.5424  | hal       | 0.443  |
| constructed           | 0.4695  | 88           | 0.5266  | sincerely | 0.4419 |
| dangerous             | 0.4567  | cautious     | 0.5199  | creditor  | 0.4372 |
| avg                   | 0.51156 | avg          | 0.54424 | avg       | 0.4449 |

| Similar words of happy |         |            |         |                     |         |
|------------------------|---------|------------|---------|---------------------|---------|
| CBOW                   |         | Skip-Gram  |         | Glove               |         |
| talked                 | 0.5173  | really     | 0.6069  | genossenschaftliche | 0.4699  |
| going                  | 0.5111  | pleased    | 0.5774  | multinational       | 0.43    |
| counterproductive      | 0.4954  | absolutely | 0.5683  | unemotional         | 0.4289  |
| indications            | 0.4952  | unable     | 0.5582  | sunday              | 0.4277  |
| revalued               | 0.4903  | talked     | 0.5451  | valid               | 0.4184  |
| avg                    | 0.50186 | avg        | 0.57118 | avg                 | 0.43498 |

| Similar words of very |         |           |         |         |         |
|-----------------------|---------|-----------|---------|---------|---------|
| CBOW                  |         | Skip-Gram |         | Glove   |         |
| negatively            | 0.4953  | wrong     | 0.569   | stir    | 0.4928  |
| extremely             | 0.4715  | positive  | 0.5597  | sheriff | 0.4746  |
| important             | 0.4698  | pretty    | 0.5571  | 526     | 0.4718  |
| methyl                | 0.4606  | winds     | 0.5435  | hawaii  | 0.4606  |
| evidence              | 0.4596  | so        | 0.5341  | 266     | 0.4451  |
| avg                   | 0.47136 | avg       | 0.55268 | avg     | 0.46898 |

We selected the calculation results for the four words *july*, *good*, *happy*, and *very* (Figure 5).

For the word *july*, both CBOW and Skip-Gram can identify words related to months, while the similar words provided by Glove are extremely absurd, even including numbers. For the word *good*, the words identified by CBOW are mostly adjectives indicating degree, such as "serious", "little", and "dangerous". However, their meanings differ greatly from that of "good", and words like "nice" or "well" do not appear. The most similar word identified by Skip-Gram is "satisfactory", which is in line with the meaning of "good". Words like "strong" and "encouraging" have positive connotations, but "88" and "cautious" have little to do with "good". In the results of Glove, there are several non-esoteric words, and only "sincerely" can express a positive meaning. For the word *happy*, in the answers of CBOW, there are almost no words expressing "happy". In the answers of Skip-Gram, "pleased" should have a higher similarity than "really", and other words have nothing to do with "happy". In the answers of Glove, there is a very long and non-existent word "genossenschaftliche". The word "sunday", as a day of rest, can be associated with "happy", but other words are far from the meaning of "happy". For the word *very* in table, the top three answers given by CBOW are all adjectives and adverbs, which are similar in part of speech to "very". "extremely" is the most similar to "very" in meaning and usage and should be ranked first, while "negatively" and "important" can only express the nature of things rather than describe degree. Among the answers provided by

Skip-Gram, "so" is very similar to "very", but its similarity is lower than that of other irrelevant words. In the answers provided by Glove, there are two numbers and three unrelated words, and its performance is poor.

Overall, in the KNN evaluation, CBOW has a better ability to identify approximate words, Skip-Gram has a higher cosine\_similarity, and Glove performs poorly under the KNN evaluation criteria.

### Word Similarity Golden Standard Evaluation

| word1  | word2    | standard_similarity | scaled_cosine_similarity |           |        |
|--------|----------|---------------------|--------------------------|-----------|--------|
|        |          |                     | CBOW                     | Skip-Gram | Glove  |
| proof  | fact     | 7.3                 | 5.862                    | 6.4521    | 4.4069 |
| winter | summer   | 2.38                | 6.2575                   | 7.685     | 4.1219 |
| nice   | generous | 5                   | 4.7797                   | 5.9044    | 4.921  |
| glass  | crystal  | 6.27                | 5.1968                   | 4.8593    | 4.6064 |
| happy  | glad     | 9.17                | 5.2708                   | 6.4914    | 5.785  |
| father | parent   | 7.07                | 5.4752                   | 6.8832    | 3.7409 |
| occur  | happen   | 9.32                | 6.7018                   | 7.5028    | 5.3938 |
| recent | new      | 7.05                | 6.2118                   | 6.1372    | 5.1965 |
| make   | earn     | 7.62                | 6.2499                   | 6.4396    | 5.4818 |
| bed    | blanket  | 3.02                | 5.1437                   | 4.2032    | 5.7114 |

As shown in the table 5, we selected 10 pairs of similar words.

The similarities given by CBOW are all concentrated around 5, and it cannot have a differentiated distribution like the Golden Standard. A value around 5 indicates that the model recognizes a certain relationship between word pairs, but cannot distinguish stronger or weaker relationships.

Skip-Gram can give similarity scores of 4 and 7. For the word pair "winter" and "summer", it is in the completely wrong direction. But it is very close to the Golden Standard for the word pair "bed" and "blanket". This shows that Skip-Gram will give lower values when dealing with word pairs whose semantic relationship is not particularly close, rather than making a uniform judgment.

The similarity scores given by the Glove model are all relatively low, around 4. It has a large gap for most word pairs with a similarity higher than 7. The Glove model performs poorly in the task of identifying related word pairs.

Overall, Skip-Gram is closer to the Golden Standard, while Glove performs the worst. For the words "winter" and "summer", although both words refer to seasons but represent different seasons, the Golden Standard only gives a similarity score of 2.38. However, all three models give similarity scores higher than 4, indicating that these methods do not distinguish the differences between these two seasons. For the words "occur" and "happen", which are very similar in meaning, the score of the Golden Standard reaches 9.32, but the similarities provided by the three models are all lower than 8. For other word pairs, the three models can be relatively close to the Golden Standard.

### Analogy Reasoning Evaluation

| A          | B          | C            | D              | CBOW          | Skip-Gram   | Glove        |
|------------|------------|--------------|----------------|---------------|-------------|--------------|
| complete   | completely | professional | professionally | glickman      | grown       | activating   |
| cincinnati | ohio       | tacoma       | washington     | echlin        | jem         | ipswich      |
| honorolulu | hawaii     | irving       | texas          | carlisle      | biologicals | alarming     |
| playing    | played     | falling      | fell           | exert         | carsons     | rout         |
| milwaukee  | wisconsin  | tulsa        | oklahoma       | pye           | dhaka       | locally      |
| read       | reading    | say          | saying         | enacted       | multifood   | transwestern |
| lexington  | kentucky   | stockton     | california     | parched       | loius       | sulzer       |
| egypt      | egyptian   | poland       | polish         | manufacture   | ccfp        | fondiararia  |
| slow       | slows      | estimate     | estimates      | quincy        | forecast    | asbi         |
| free       | freely     | usual        | usually        | comparability | occurs      | gaosline     |

As for the Analogical Reasoning task, all models failed to solve the analogies, achieving an accuracy of 0/39 (0%). The predicted vectors were plotted (Figure 1) for visualization, but no meaningful relationships were observed. This can be responsible for the limited training data as the Reuters corpus lacks diverse syntactic structures and relationships. Nevertheless, the experimental small embedding dimension (64) may also contribute to the failure, as it may not capture the complexity of relationships effectively for encoding.